COURS WEB

Created by **Corentin LATAPPY**
Inspired by **Cyril BESLAY**

QUI SUIS-JE ?

- Corentin Latappy
- Doctorant CIFRE au **LaBRI** & **Promyze**
- Dev full-stack en JS & TS
- **corentin.latappy@labri.fr**

promyze

- Startup française créée en 2016
- Plateforme de partage des bonnes pratiques entre développeurs

PLAN DU COURS

- Cours 1 : Introduction et bases de JavaScript
- Cours 2 : Développement côté serveur avec Express
- Cours 3 : Développement côté client avec VueJS
- Cours 4 : Composants et Routeur
- Cours 5 : Socket.io

PLAN DES TPS

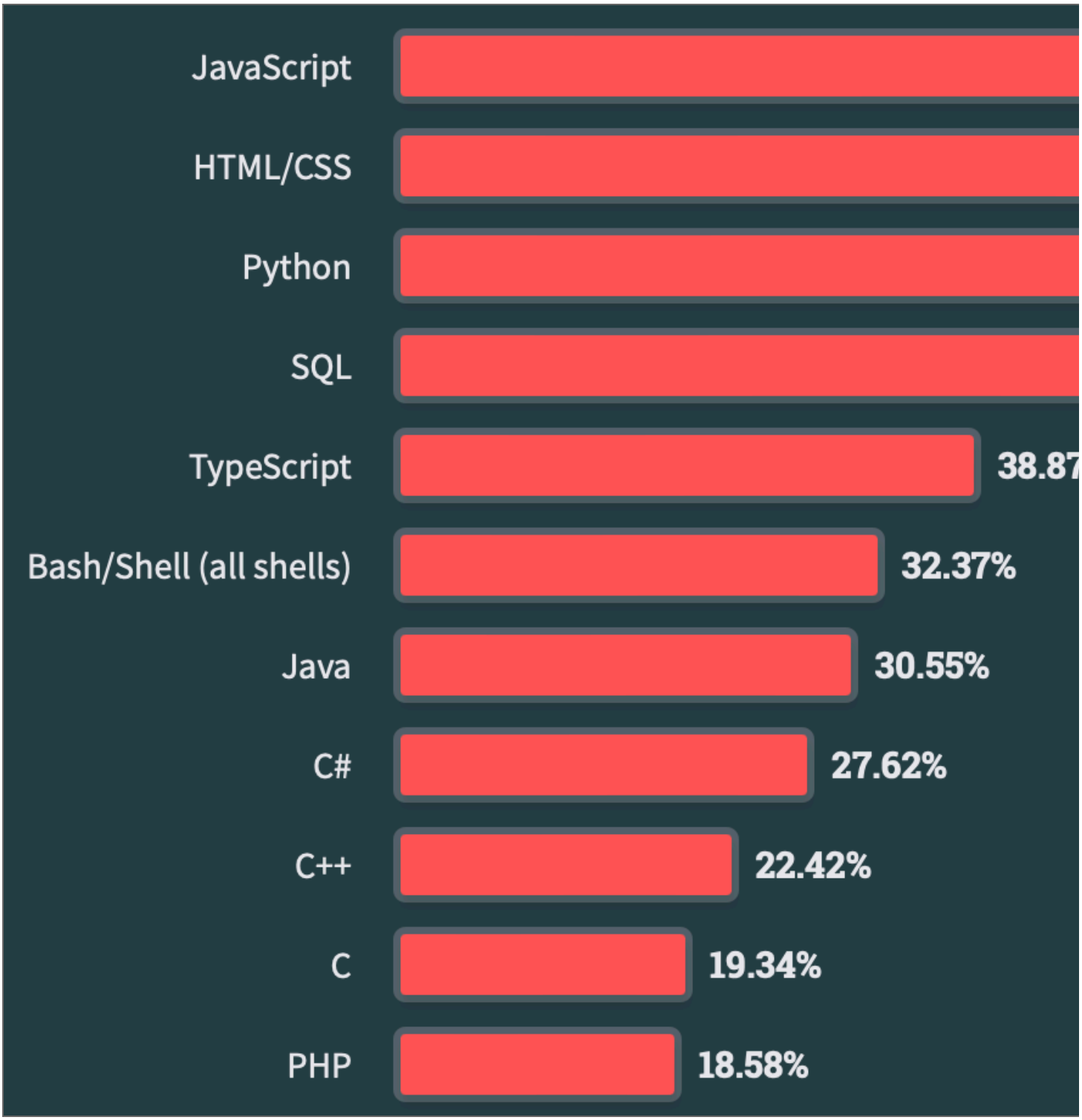
- TP 1 : Site Web Météo dynamique
- TP 2 : PHE - Partie Serveur
- TP 3 : PHE - Partie Client
- TP 4 : PHE - Composants et Routes
- TP 5 : PHE - Communication et Refactoring

ACCÈS AUX SUPPORTS

GitHub

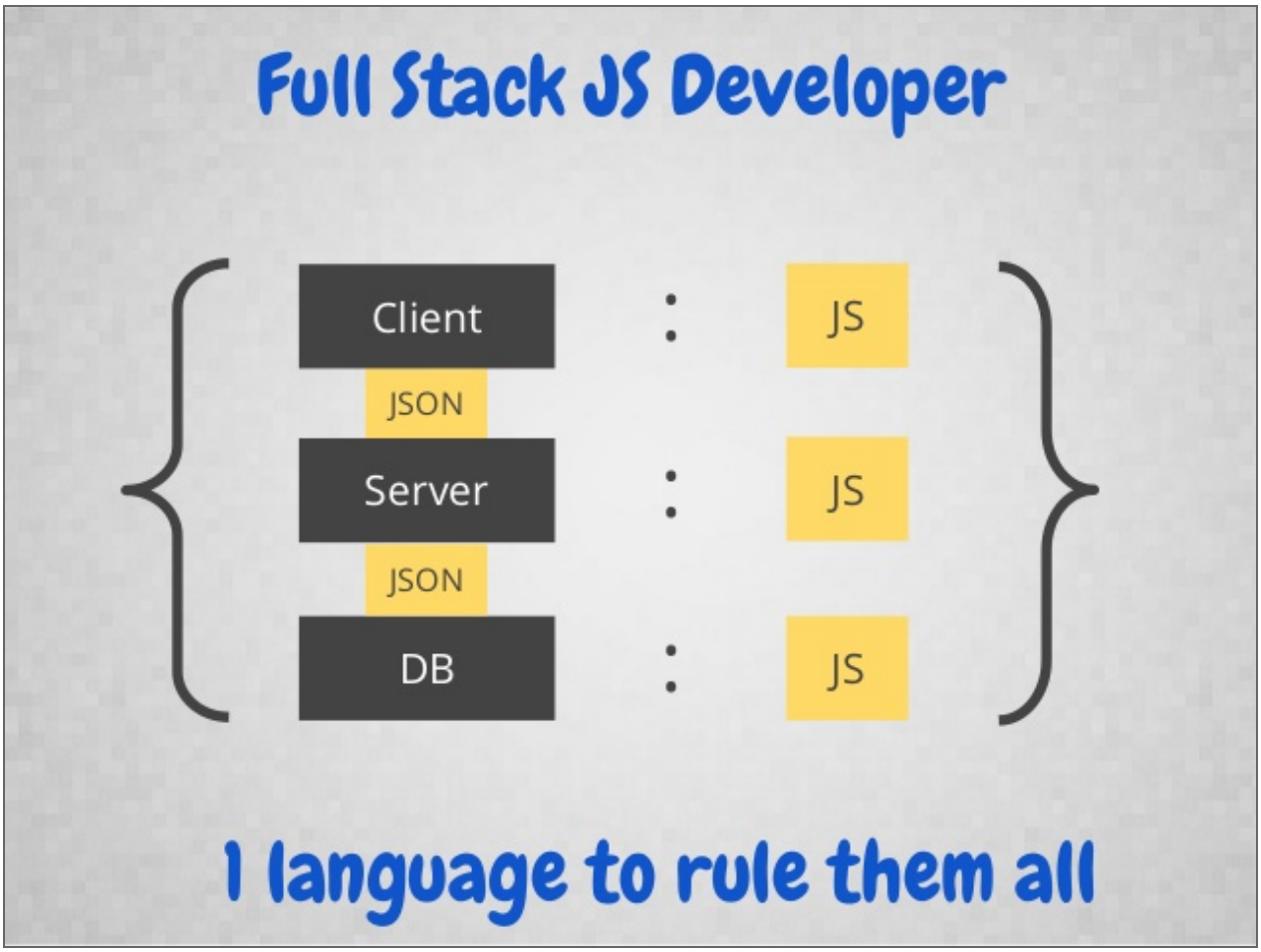
POURQUOI JAVASCRIPT ?

Langage très utilisé et en forte progression



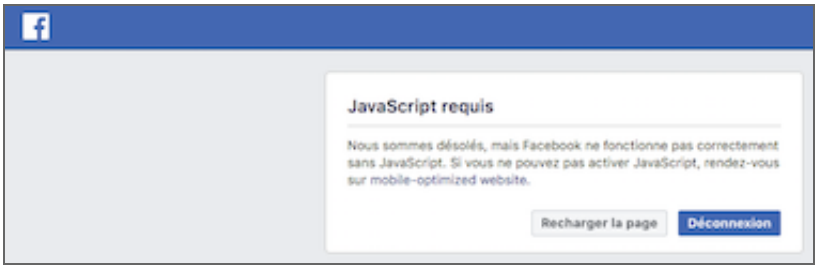
https://survey.stackoverflow.co/2023

Langage Full Stack



- Frontend
- Serveur
- Mobile
- Jeu

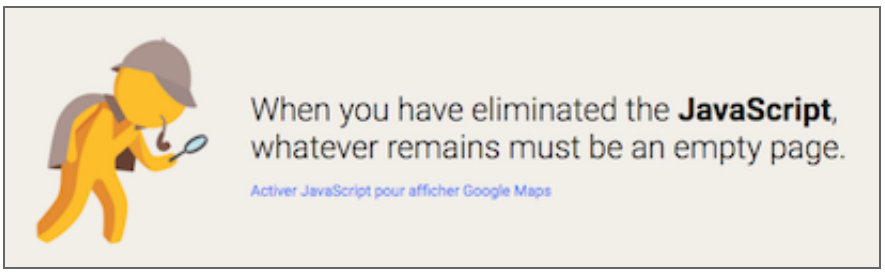
INTERNET SANS JAVASCRIPT



Pour pouvoir utiliser Gmail en affichage standard, vous devez activer JavaScript. A
standard, activez JavaScript en modifiant les options du navigateur, puis réessayez

Pour utiliser l'affichage HTML simplifié dans Gmail, qui ne nécessite pas que JavaS

Si vous souhaitez afficher Gmail sur un téléphone mobile ou un appareil similaire,



ÉCOSYSTÈME

LE LANGUAGE JAVASCRIPT

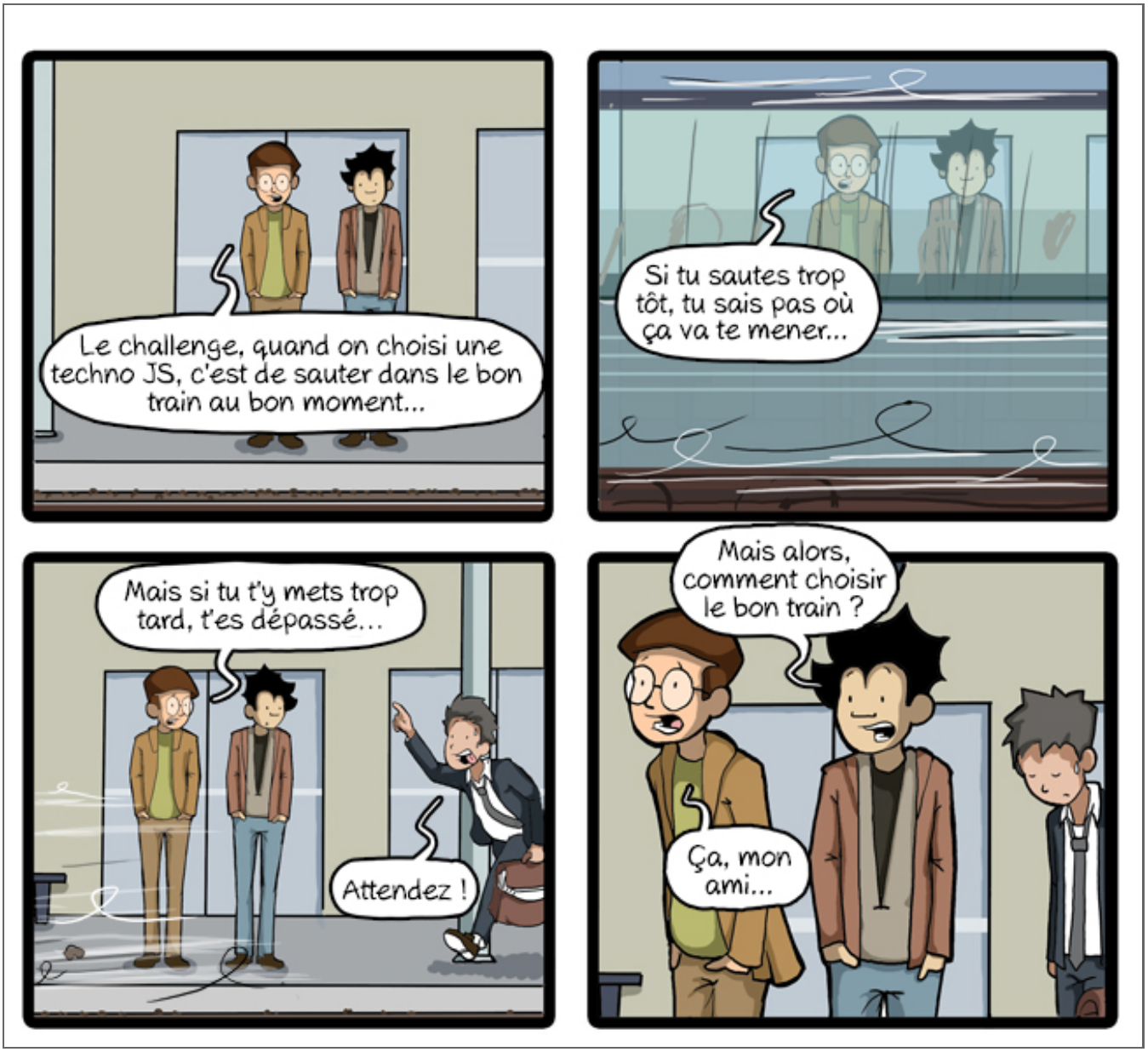
```
    // Set variables
    let myBankBalance = 0;
    let output = "";

    // Do the 'for' loop
    for (myBankBalance = 0; myBankBalance <=
3; myBankBalance++) {
      if  (myBankBalance === 1) {
        continue;
      }
      output += "My bank balance is now $" +
myBankBalance + "<br>";
    }

    // Output results to the above HTML
element
    document.getElementById("msg").innerHTML
= output;
```

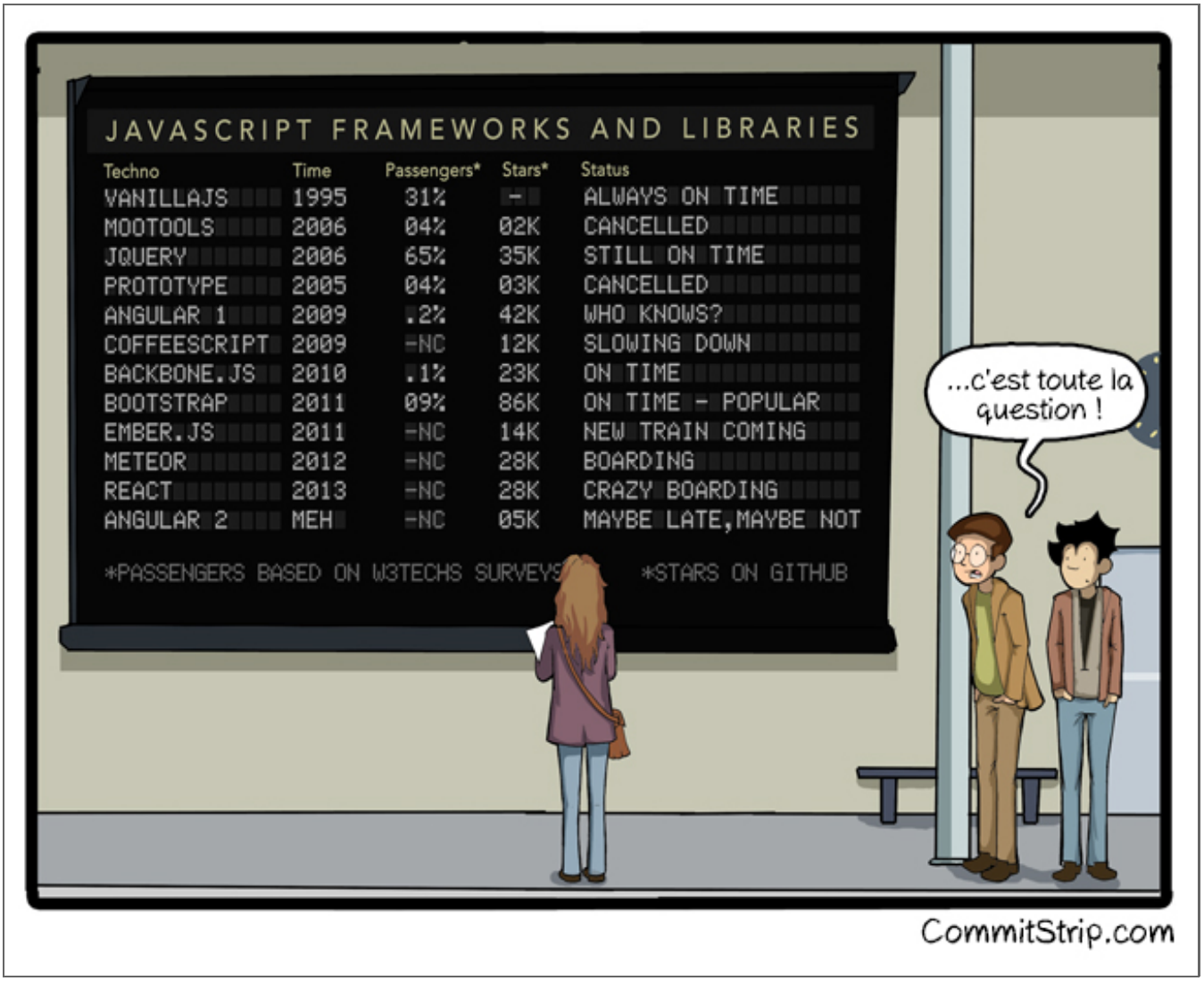
ECMASCRIPT

- **Standard** international qui définit JavaScript
- Apporte de nouvelles fonctionnalités
- Non compatible avec tous les navigateurs
- ES5, ES6, ES2017, ...

FRAMEWORKS

- Bibliothèques qui permettent de faciliter le développement, donner un cadre et des outils
- Beaucoup de frameworks disponibles
- Environnement évoluant très vite
- 3 acteurs majeurs : Angular, React, Vue.js


Le challenge, quand on choisi une techno JS, c'est de sauter dans le bon train au bon moment...
Si tu sautes trop tôt, tu sais pas où ça va te mener...
Mais si tu t'y mets trop tard, t'es dépassé...
Attendez !
Mais alors, comment choisir le bon train ?
Ça, mon ami...



CommitStrip.com

BACKEND

Aujourd'hui, JavaScript n'est pas seulement utilisé dans le web mais aussi pour des applications lourdes (Discord, WhatsApp Desktop, ...) et pour des serveurs.
Même syntaxe, mais librairies et concepts différents.

OUTILS

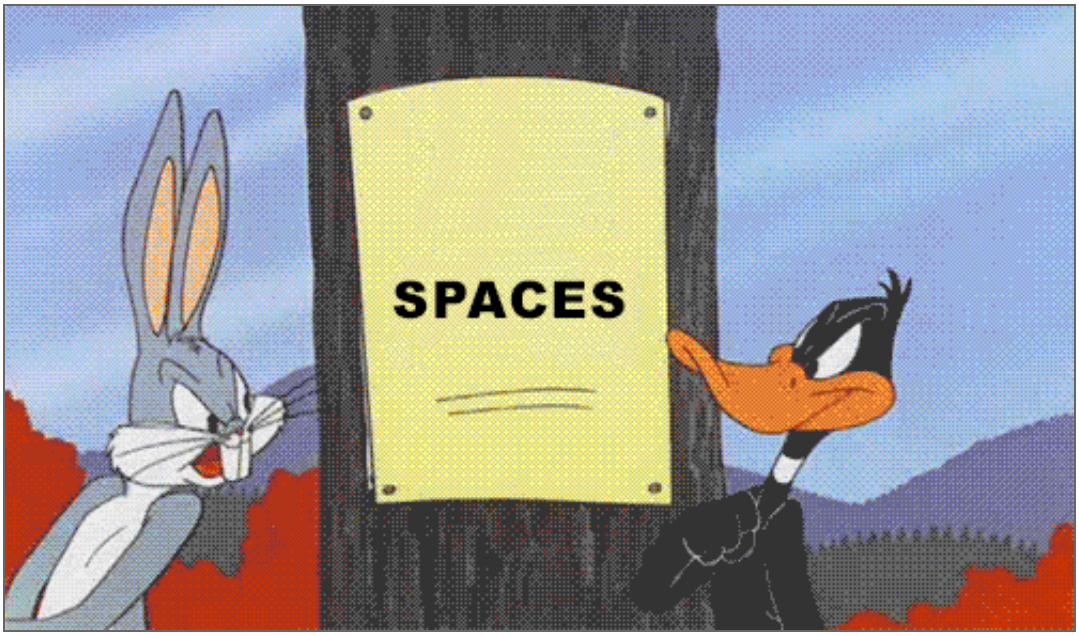
- Navigateur (Chrome, Firefox, ...)
- Editeur (WebStorm, VSCode, Atom, ...)
- Terminal
- Node Package Manager - NPM

LANGUAGE JAVASCRIPT

OÙ DÉVELOPPER ?

- Console Chrome
- Page web HTML
- Serveur NodeJS

LES NORMES D'ÉCRITURE



L'idée de base est que "les développeurs d'un même projet codent de la même manière".
Pour ce cours et les TPs, ça n'a pas d'importance. Restez constants.

LANGUAGE INTERPRÉTÉ

- Pas de compilation.
- Chaque marque a son moteur (Microsoft, Google, Apple, Mozilla).
- Language visible dans le navigateur (exemple).
- Scripts exécutés chez le client.

HELLO WORLD !

```
alert('Hello World !');
```

VARIABLES

- let
- const
- (var)
- camelCase

```
var messageToSend = 'Hello World !';
let author = 'Cyril';
const country = 'FR';
author = 'Kevin';
alert(messageToSend);
```

TYPAGE DYNAMIQUE

- Une variable peut contenir **tout type de donnée**.
- Le type de donnée dans une variable peut changer au cours du temps.

```
let studentNumber = 150;
console.log(studentNumber);
studentNumber = '150';
console.log(studentNumber);
```

- null : signifie valeur inconnue
- undefined : Variable non définie

```
let studentNumber;
console.log(studentNumber); // undefined
studentNumber = null;
console.log(studentNumber); // null
```

type of pour connaître le type d'une variable

```
typeof undefined // "undefined"
typeof 0 // "number"
typeof true // "boolean"
typeof "foo" // "string"
typeof {'foo' : 'fact'} // "object"
typeof null // "object"
typeof alert // "function
```

COMPARAISONS

Attention au type

```
    let studentNumber = 150;
    let studentNumberExpected = '150';
    studentNumber == studentNumberExpected //
true
    studentNumber === studentNumberExpected
// false
```

CONDITIONS

if

```
    const year = prompt('In which year was
ECMAScript-2015 specification published?',
'');

    if (year == 2015) {
      alert('You guessed it right!');
    } else {
      alert('How can you be so wrong?');
    }
```

switch

```
const a = 2 + 2;

    switch (a) {
      case 3:
        alert('Too small');
        break;
      case 4:
        alert('Exactly!');
        break;
      case 5:
        alert('Too large');
        break;
      default:
        alert("I don't know such values");
    }
```

BOUCLES

- for
- while (do while)
- break
- continue

```
for (let i = 0; i < fruits.length; i++) {
  if (fruits[i] === 'Banana') {
    continue;
  }
  console.log(fruits[i]);
}

while (i < 10) {
  console.log('The number is ' + i);
  i++;
  if (i > 7) {
    break;
  }
}
```

FONCTIONS

```
function checkAge(age, legalAge = 18) {
  if (age >= legalAge) {
    return true;
  } else {
    return 'Got a permission from the
parents ?';
  }
}

let response = checkAge(19);
console.log(response); // true
response = checkAge(19, 21);
console.log(response); // false
```

DEUX SYNTAXES

```
    // Function classique
    function checkAge(age, legalAge = 18) {
      return age >= legalAge;
    }

    // Function arrow
    const checkAge = (age, legalAge = 18) =>
(age >= legalAge);

    const cities = data.filter(city =>
(city.population > 500000));
```

TABLEAUX

- Facile à manipuler
- Nombreuses fonctionnalités portées par l'objet (length, push, pop, shift)
- Parcours de 0 à length - 1

```
    let fruits = ['Apple', 'Orange', 'Pear'];

    for (let i = 0; i < fruits.length; i++) {
      console.log(fruits[i]);
    }

    fruits.push('Citron');

    console.log(fruits); // ['Apple',
'Orange', 'Pear', 'Citron']
    console.log(fruits.length); // 4

    console.log(fruits.shift()); // 'Apple'
    console.log(fruits); // ['Orange',
'Pear', 'Citron']
```

MANIPULATION DE TABLEAUX

forEach

```
    const fruits = ['Apple', 'Orange',
'Pear'];
    fruits.forEach(fruit =>
console.log(fruit)); // Apple, Orange, Pear
```

MANIPULATION DE TABLEAUX

map

```
const numbers = [0, 1, 2, 3];
const squares = numbers.map(number =>
number * number);
console.log(squares); // [0, 1, 4, 9]
```

MANIPULATION DE TABLEAUX

filter

```
const numbers = [0, 1, 2, 3, 4, 5];
const evens = numbers.filter(number =>
number % 2 === 0);
console.log(evens); // [0, 2, 4]
```

OBJETS

Un objet est un type complexe capable de stocker plusieurs informations.
L'objet a un ensemble de clés/valeurs appelé propriétés.
L'accès à une propriété se fait via `objet.property` ou `objet['property']`.

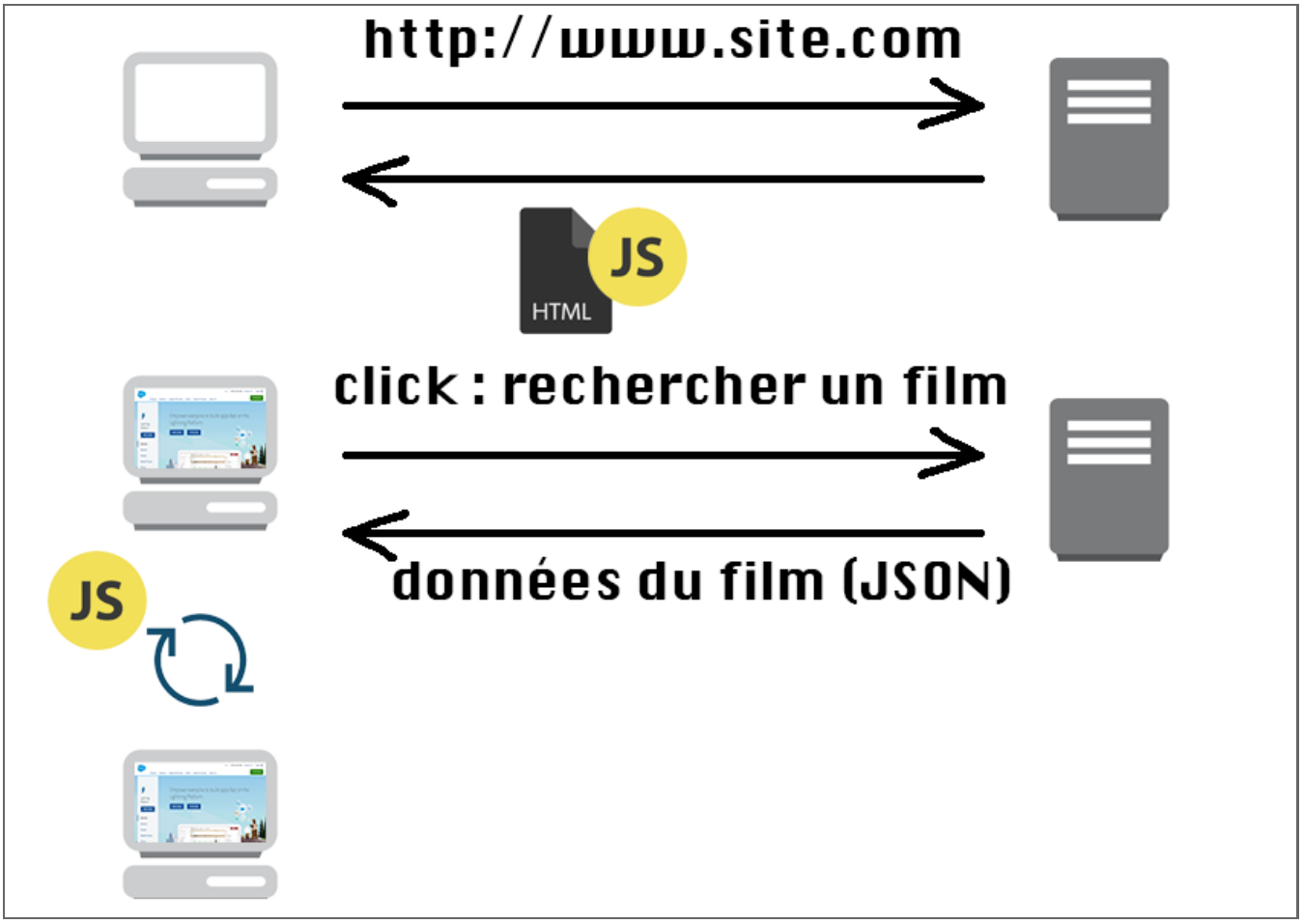
Attention, les objets sont des références et non des valeurs.

```
    const student = { name: 'Clark', age: 23
};
    const admin = student; // copy the
reference

    console.log(student.name); // 'Clark'
    admin.name = 'Superman';
    console.log(student.name); // 'Superman'
```

THIS

```
    const user = {
      name: 'John',
      age: 30,
      sayHello() {
        return 'Je suis ' + this.name + ' et
j\'ai ' + this.age + ' ans.';
      }
    };
```

ÉCHANGE CLIENT-SERVEUR



HYPERTEXT TRANSFERT PROTOCOL

HTTP est un **protocole de communication** client-serveur développé pour le web.
GET, POST, PUT, DELETE sont des **verbes** HTTP

COMMENT APPELER UN SERVEUR ?

2 méthodes :

- XMLHttpRequest
- fetch (ES6)

XMLHTTPREQUEST

```
const req = new XMLHttpRequest();
req.open('GET', url);
req.send();
```

FETCH

```
fetch(url)
  .then(() => {
    // Do something
  });
```

JSON

Format d'échange de données structurées

```
    {
      "book": {
        "title": "Moby Dick",
        "author": "Herman Melville",
        "reviews": [
          {"reviewer": "Mark234",
"rating_stars": "4.5"},
          {"reviewer": "Frank12",
"rating_stars": "3.5"},
          {"reviewer": "NotPatrick",
"rating_stars": "4"}
        ]
      }
    }
```

PROMISE

Une promesse représente une valeur qui peut être disponible maintenant, plus tard ou jamais.

```
    const promise = new Promise((resolve,
reject) => {
      setTimeout(function() {
        resolve();
      }, 300);
    });

    promise.then(() => console.log('arrivé
2ème :(', ));

    console.log('arrivé 1er :)');
```

PROMISE

Beaucoup de fonctions travaillent en asynchrone, donc avec un système de promesse.
Exemples de fonctions I/O :

- fetch (HTTP)
- fs (filesystem)
- database querying

CHAINER LES PROMESSES

```
fetch('./file.json')
  .then(response => response.json())
  .then(data => console.log(data))
```

ASYNC / AWAIT

Introduit depuis ES8
Même résultat que les promesses mais plus lisible

ASYNC / AWAIT

```
    const response = await
fetch('./file.json');
    const data = await response.json();
    console.log(data);
```

RESSOURCES EXTERNES

https://javascript.info/
https://www.learn-js.org/

ECMASCRIPT

ECMASCRIPT 5, 6, 7, 8...

EcmaScript définit les standards et nouveautés de développement
Problème : les navigateurs et moteurs JavaScript mettent du temps à s'adapter
Une notion de compatibilité entre en jeu

ECMASCRIPT COMPATIBILITÉ

Tableau de compatibilité des différents navigateurs et moteurs

Tableau de Compatibilité

ES5

- Fonctions Array forEach, map, filter
- JSON.parse() et JSON.stringify()
- getters et setters sur les objets

ES2015 (ES6)

- Paramètre par defaut
- String literals
- Arrow functions
- Promise
- Const and Let
- Classes and super

ES2023 (ES14)

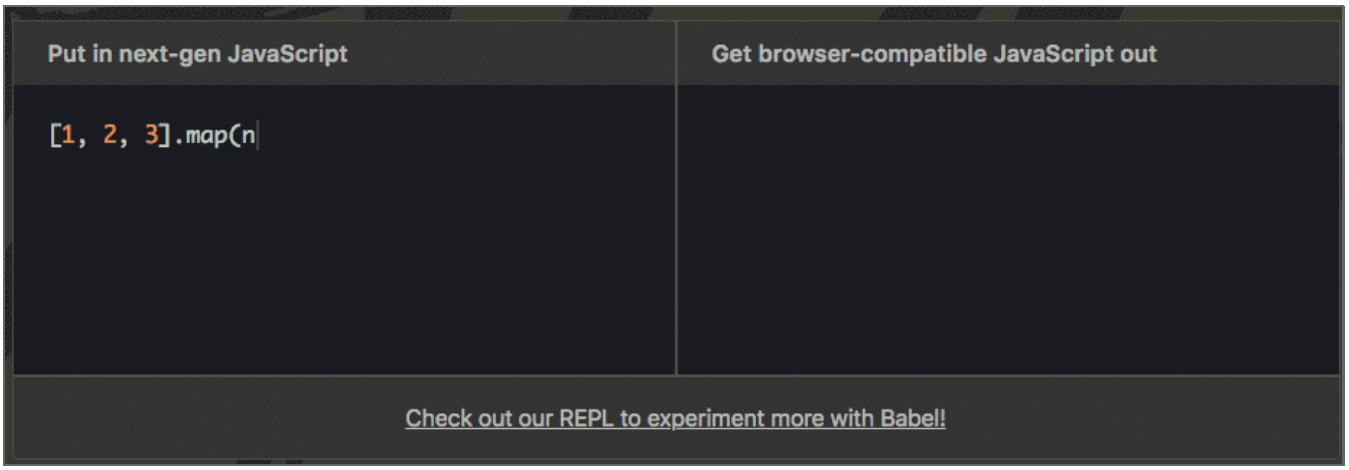
Pas encore stable, pas encore compatible
Ne partez pas trop tôt

BABEL & WEBPACK

Comment faire pour utiliser ES6/ES7 en s'assurant d'être **compatible** avec le plus grand nombre ?
La **transpilation** permet de tranformer du code "nouveau" en code "ancien".
Dans notre cas, ECMAScript vers JavaScript.
Il est donc possible d'écrire du code avec des fonctionnalités modernes tout en restant compatible avec tous les navigateurs.

BABEL JS

Un outil de transpilation majoritairement utilisé



WEBPACK

Outil qui permet :

- d'executer des transformations sur des fichiers
- d'agréger des fichiers
- une grande configuration/personnalisation

ES6
BABEL
webpack
my search

NODE JS ET EXPRESSJS

NODEJS

Environnement d'éxécution de JavaScript côté serveur
Basé sur le moteur **V8**
Version Long Time Supported : 18

NODE PACKAGE MANAGER

Gestionnaire de paquets officiel de NodeJS
Permet l'installation et la gestion de dépendances (librairies) pour un projet

NODE PACKAGE MANAGER

La configuration se fait via un fichier **package.json**

```
{
  "name": "phe-backend",
  "version": "1.0.0",
  ...
  "dependencies": {
    "express": "^4.17.1",
    "sqlite3": "^5.1.6"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

NODE PACKAGE MANAGER

Deux commande principales :

- `npm i (npm install)`
- `npm i paquet`

EXPRESSJS

Bibliothèque légère pour développer des applications NodeJS
Gestion des routes pour une API HTTP

EXPRESSJS

```
import express from 'express';
const app = express();
app.get('/', function(req, res) {
  res.send('Bienvenue');
});
app.listen(8080);
```

EXPRESSJS

```
app.get('/hello/:name', function(req, res) {
  const name = req.params.name;
  res.send('Bienvenue ' + name);
});
```

NOTION DE MODULE

Pour mieux structurer notre code, nous avons besoin de le séparer en plusieurs fichiers.
Nous pouvons importer des modules internes ET externes

```
    import http from 'http'; // import
externe (coeur node)
    import express from 'express'; // import
librairie externe (node_modules)
    import utils from './utils'; // import
interne (dossier utils)
```

```
    export function dateFrancais() {
      return new
Date().toLocaleDateString('fr-FR');
    }
```

```
import dateModule from './dateModule';
console.log(dateModule.dateFrancais());
```

FRAMEWORKS JAVASCRIPT

POURQUOI UTILISER UN FRAMEWORK ?

- Cadre de développement
- Développement plus rapide
- Structuration du code et réutilisation
- Faciliter la manipulation du DOM

> *N'utilisez pas un framework JavaScript pour développer une simple page.*

SINGLE PAGE APP

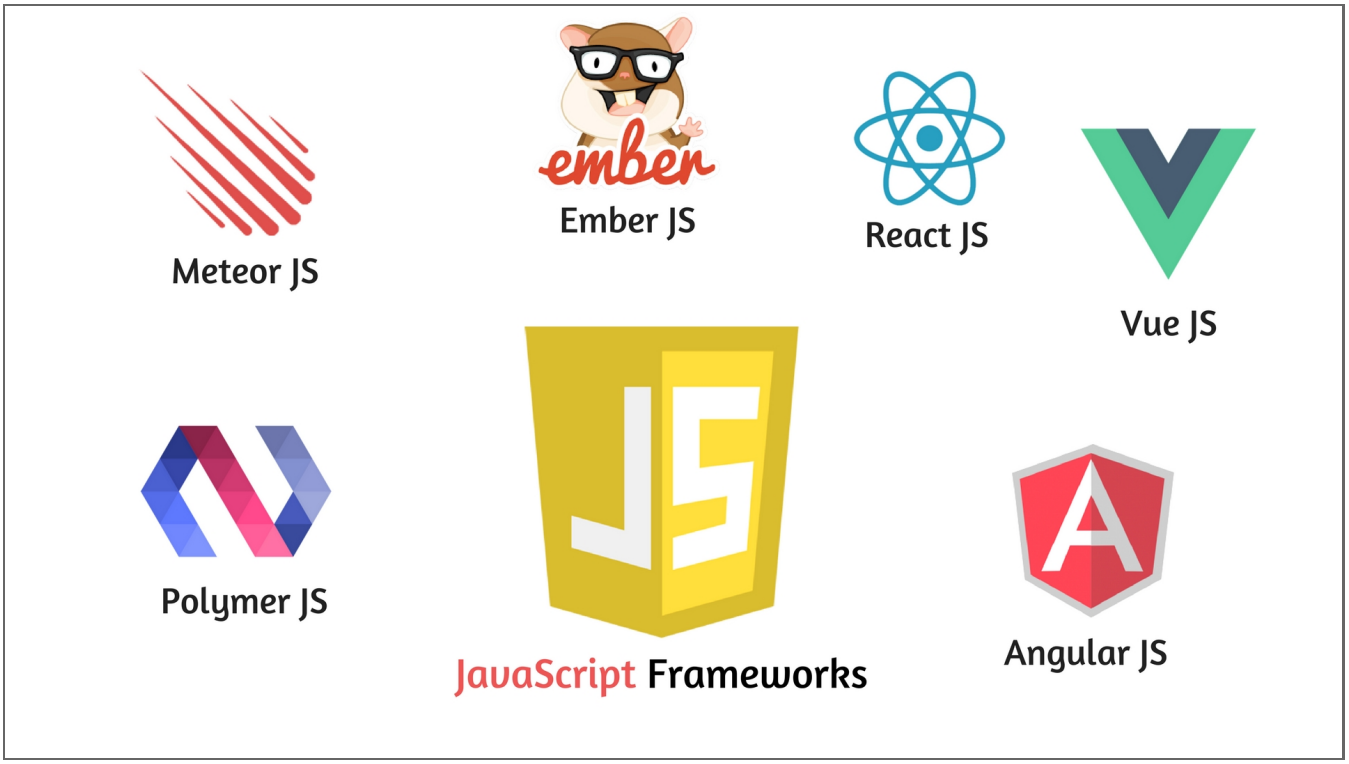
Principe : éviter le chargement d'une nouvelle page à chaque action afin de **fluidifier** l'expérience utilisateur
Optimisation nombre d'appels HTTP

SINGLE PAGE APP

Poids des pages augmente mais :

- Possibilité de faire du rendu progressif
- Les débits Internet augmentent

QUEL FRAMEWORK ?



QUEL FRAMEWORK ?

	VueJS	Angular	ReactJS	EmberJS
Courbe d'apprentissage	😃	😐	😐	😐
Efficacité	😃	😐	😐	😐
Maintenance	😐	😟	😟	😃
Ecosystème	😐	😃	😐	😐
Popularité	😐	😐	😃	😟
Performance	😃	😃	😃	😟
Niveau requis	😃	😟	😐	😐

VUEJS

[Vue.js]

VUEJS

- Développer rapidement des interfaces utilisateurs
- Prise en main progressive : coeur simple mais bibliothèques et écosystème complets
- Principe de découpage en composants réutilisables et autonomes
- Très bonne documentation

VUEJS

Créé en 2014 par Evan You
Open source : **https://github.com/vuejs/vue**
Utilisé par Netflix, Adobe, Alibaba et Gitlab.

Comparaison avec React :

- beaucoup de similitudes (dom, components)
- template JSX vs Mustache
- JSX + ES6 vs apprentissage rapide
- React Native (applications mobiles)

Comparaison avec Angular (Angular 2 et +) :

- VueJS inspiré d'Angular
- TypeScript
- complexité
- stricte vs flexibilité
- poids

PREMIÈRE PAGE VUEJS

Utilisation de VueJS en composant monofichier

---

3 parties dans chaque composant :

- template (HTML)
- script (logique)
- style (CSS)

VUEJS EN COMPOSANT MONOFICHIER

```
<template>
  < HTML />
</template>
<script>
  JavaScript;
</script>
<style>
  .CSS {}
</style>
```

PARTIE TEMPLATE

```
    <template>
      <input type="text" v-
model="etudiant.nom">
      <button @click="ajouter"></button>
      {{ etudiant.nom }}
    </template>
```

PARTIE SCRIPT

```
<script>
  import { reactive, ref } from 'vue';

  const etudiants = ref([]);
  const etudiant = reactive({ nom: 'John'
});

  const ajouter = () => {
    etudiants.value.push(etudiant.nom);
  };
</script>
```

PARTIE STYLE

S'applique uniquement sur le composant / la page

```
<style>
  div.figure {
    float: right;
    width: 30%;
  }
  .title {
    color: blue;
  }
</style>
```

BINDING 2-WAYS

Lien entre template (affichage) et script (données).
Si l'utilisateur modifie l'affichage, la donnée est modifiée.
Si la donnée est modifiée, l'affichage se met à jour.

BINDING 2-WAYS

Utilisation de Mustache

```
<div id="app">
  {{ message }}
</div>
```

Utilisation de v-model

```
<div id="app">
  <input type="text" v-model="message">
</div>
```

DIRECTIVES

Attributs spécifiques à VueJS
v-if permet d'afficher ou non un bloc selon l'état d'une variable

```
<p v-if="loggedIn">Hello !</p>
<p v-else="">Please log in</p>
```

DIRECTIVES

v-for permet d'afficher une liste d'élements

```
    <ul>
      <li v-for="movie in movies"
:key="movie.id">
        {{movie.name}} - {{movie.date}}
      </li>
    </ul>
```

MÉTHODES

Fonctions contenant la logique, permettant de manipuler les données

```
<script>
  import { ref } from 'vue';

  const nbEtudiants = ref(0);

  const incrementeEtudiants = () => {
    nbEtudiants.value++;
  };
</script>
```

EVÈNEMENTS

Actions ou interactions utilisateurs
Le plus souvent, le but va être de connecter des évènements à des méthodes.
Utilisation du mot-clé @

EVÈNEMENTS

```
    <template>
      <button
@click="applaudir">Applaudir</button>
      <p>Vous avez reçu {{applaudissements}}
applaudissements.</p>
    </template>

    <script>
      import { ref } from 'vue';

      const applaudissements = ref(0);
      const applaudir = () => {
        this.applaudissements.value++;
      };
    </script>
```

PROPRIÉTÉS CALCULÉES

Pour les traitements de donnée lourds avant affichage
L'idée est d'éviter de mettre ces traitements dans le template
Utilisation du mot-clé **computed**

PROPRIÉTÉS CALCULÉES

```
<script>
  import { computed, ref } from 'vue';

  const nbEtudiantsGroupe1 = ref(10);
  const nbEtudiantsGoupe2 = ref(20);

  const somme = computed(() =>
    nbEtudiantsGroupe1 +
nbEtudiantsGoupe2
  );
</script>
```

CYCLE DE VIE

Chaque composant/page a un cycle de vie (création, monté, détruit).
Il est possible de "s'accrocher" à des moments du cycle pour effectuer des actions.

```
<script>
import { onMounted } from 'vue';

onMounted(() => {
  console.log(`the component is now mounted.`);
});
</script>
```

WEB COMPONENTS

Un Composant est un morceau de page web autonome et réutilisable

WEB CUSTOM COMPONENTS

Basé sur Custom Elements, Shadow DOM, Template, des normes HTML

VUEJS COMPONENTS

Similaire à Web Custom Components
Permet de découper l'application en plusieurs pages et chaque page en plusieurs éléments

VUEJS COMPONENTS

Avec notre approche composant monofichier, chaque fichier .vue est un composant.
Le fichier App.vue représente la page parente.

VUEJS COMPONENTS

Exemple composant CharacterItem.vue :

```
    <template>
      <h4>
        {{ character.name }}
      </h4>
      <p>
        Couleur des yeux : {{
character.eyeColor }}
      </p>

      <button @click="emit('deleteCharacter',
character)">
        Supprimer
      </button>
    </template>

    <script>
      defineProps(['character']);
      const emit =
defineEmits(['deleteCharacter']);
    </script>
```

DATA DOWN, EVENT UP

Transférer des données à un composant fils se fait via des propriétés

```
<template>
  <h4>
    {{ character.name }}
  </h4>
  <p>
    Couleur des yeux : {{
character.eyeColor }}
  </p>
</template>

<script>
  defineProps(['character']);
</script>
```

DATA DOWN, EVENT UP

Remonter des actions à un composant père se fait via des évènements

```
    <template>
      <button @click="emit('deleteCharacter',
character)">
        Supprimer
      </button>
    </template>

    <script>
      const emit =
defineEmits(['deleteCharacter']);
    </script>
```

VUEJS COMPONENTS

Utilisation composant :

```
    <template>
      <characteritem v-for="character in
characters" :key="character.name"
:character="character"
@deletecharacter="deleteCharacter">
</characteritem>
    </template>

    <script>
      import { ref } from 'vue';
      import CharacterItem from
'./components/CharacterItem';

      const characters = ref([
        { name: 'Toto', eyeColor: 'brown' },
        { name: 'Toto', eyeColor: 'blue' },
      ]);

      const deleteCharacter = (character) =>
{};
    </script>
```

VUEJSROUTER

ROUTER / ROUTAGE ?

Dans un site web classique, chaque page a une URL différente :

- https://amazon.fr/panier
- https://amazon.fr/paiment
- https://amazon.fr/article-27

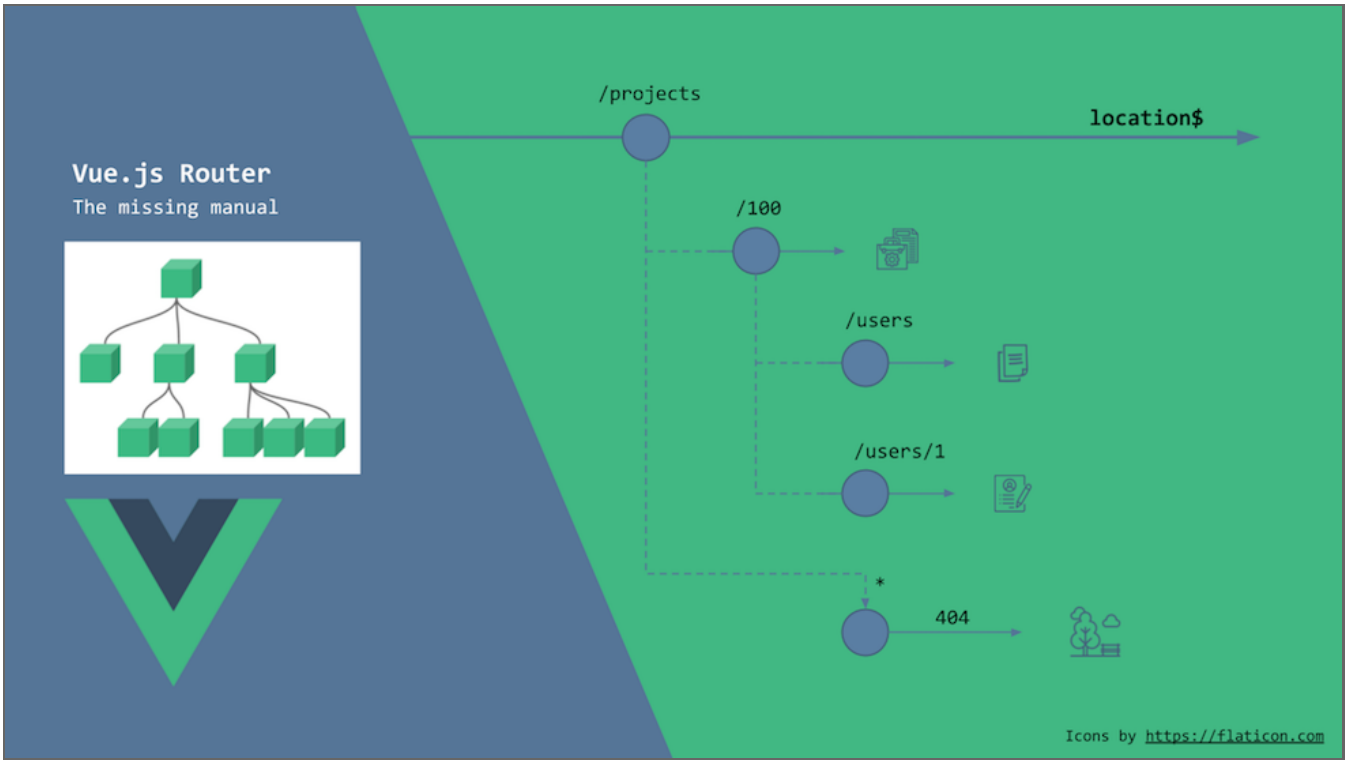
ROUTER / ROUTAGE ?

Dans un site SPA, tout est sur la première page. Se posent les problèmes suivants :

- SEO (moteurs de recherche)
- Historique
- Partage de liens

ROUTER / ROUTAGE ?

La solution est le router qui va faire le lien entre l'url et les pages.



VUE-ROUTER

La librairie proposée avec VueJS pour gérer les routes.
https://router.vuejs.org
https://github.com/vuejs/router

VUE-ROUTER

Trois étapes :

- Définition des routes
- Utilisation des routes dans les templates
- Utilisation des routes dans la logique

VUE-ROUTER

Définition des routes : attacher des URLs à des composants

```
    routes: [
      { path: '/', name: 'Home', component:
Home },
      { path: '/race', name: 'Race',
component: Race },
      { path: '/login', name: 'Login',
component: Login }
    ]
```

VUE-ROUTER

Utilisation des routes dans les templates

```
    <router-link to="/login">Se connecter</router-link>
    <router-link to="/race">Course</router-link>
```

VUE-ROUTER

Utilisation des routes dans la logique

```
    // chaine de caractère représentant un
chemin
    router.push('/login');

    // route nommée
    router.push({ name: 'Race' });
```

VUE-ROUTER

Routes dynamiques

```
routes: [
  { path: '/utilisateur/:id', component: User, name: 'user' }
]

router.push({ name: 'user', params: { id: 123 } })
```

VUE-ROUTER

- Chaque route est liée à un composant (une page)
- Chaque page peut contenir plusieurs composants
- Lorsque l'utilisateur clique sur un lien de navigation, vue-router modifie la page pour afficher le nouveau composant

VUEX

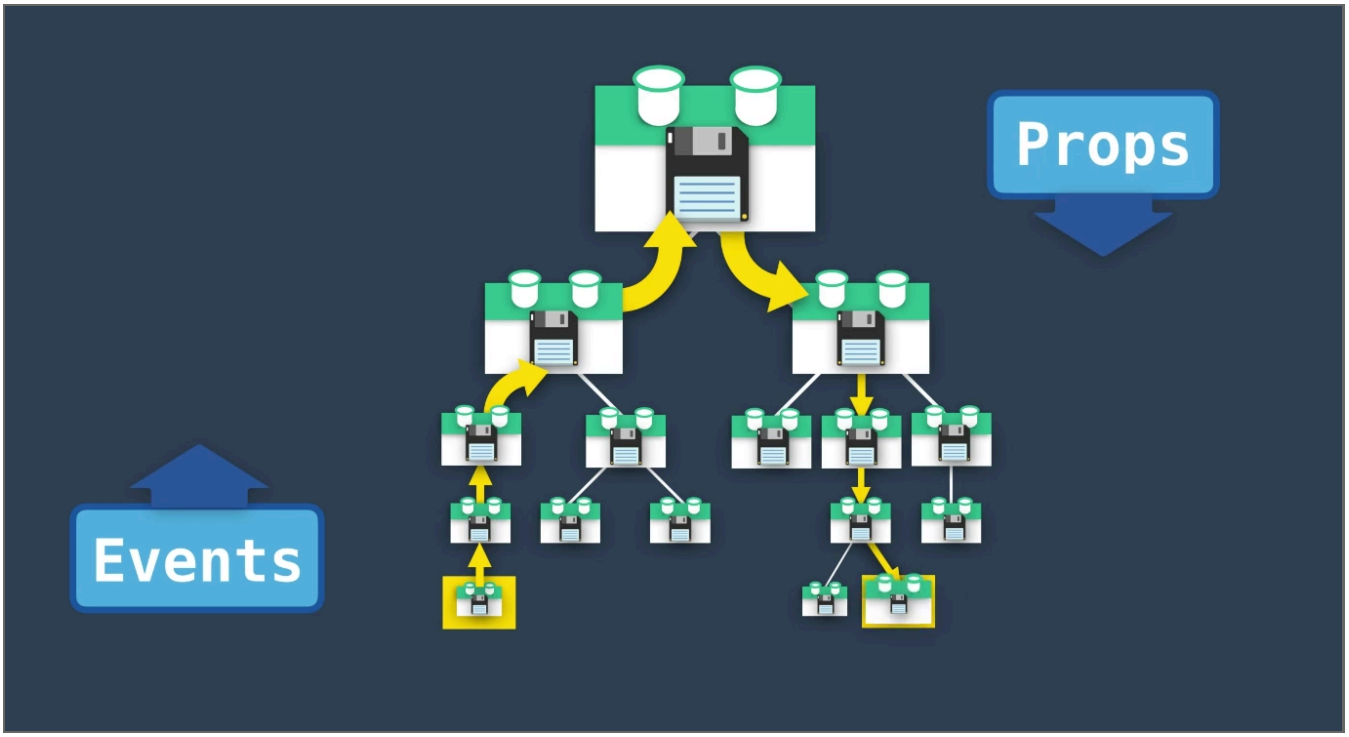
VUEX

VueX est un gestionnaire d'état
https://vuex.vuejs.org/

VUEX

Sans gestionnaire d'état, toute donnée modifiée dans un composant est difficile à partager au sein de toute l'application.

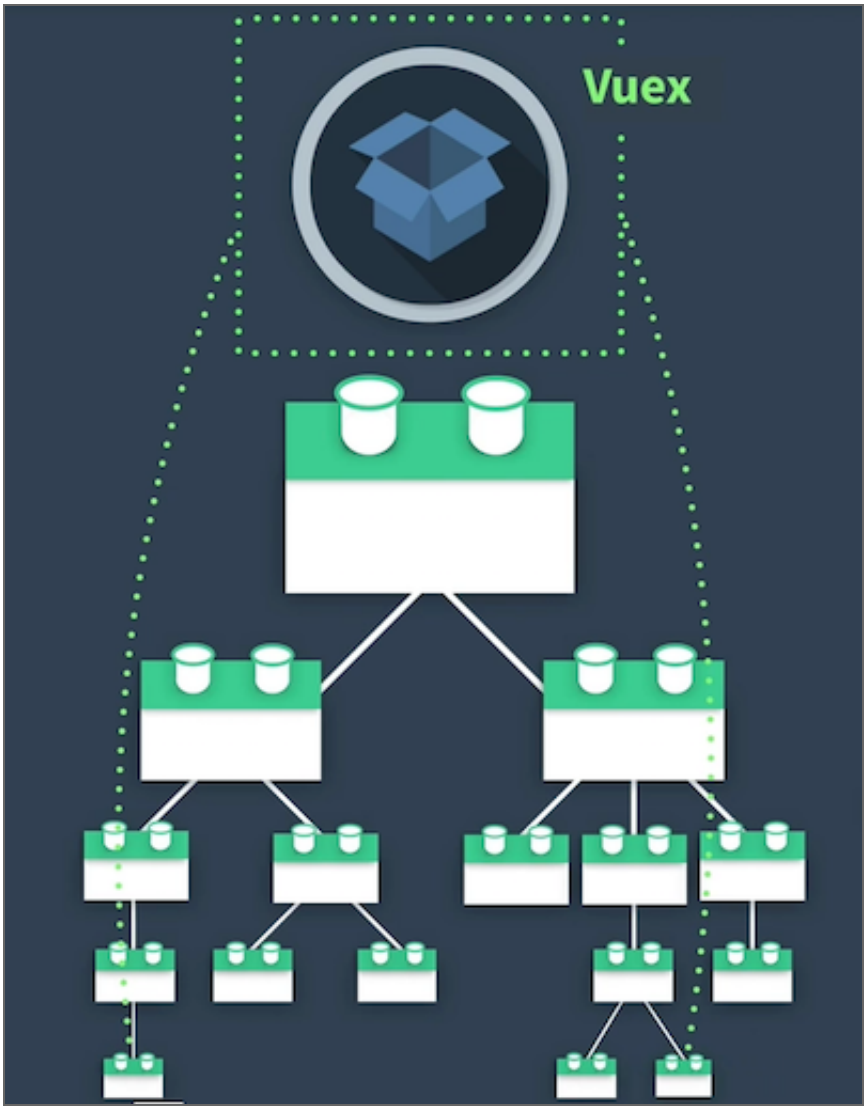
VUEX



VUEX

Avec VueX, il est possible de gérer un "store" (modèle de stockage) contenant des données/états.

VUEX



VUEX

Exemples de besoins :

- stocker l'état connecté/déconnecté d'un utilisateur
- stocker le panier d'un client
- stocker préférences utilisateur
- alléger les appels backend

VUEX

```
const app = new Vue({
  data: {
    …
  },
  methods: {
    …
  },
  computed: {
    …
  }
})
```

```
const store = new Vuex.Store({
  state: {
    …
  },
  mutations: {
    …
  },
  actions: {
    …
  },
  getters: {
    …
  }
})
```

VUEX - STATE

L'état "state" est la liste des données à sauvegarder
Equivalent à **data** dans un composant

```
state: {
  count: 0
},
```

VUEX - MUTATIONS

Les mutations modifient les états

```
mutations: {
  INCREMENT: function(state) {
    state.count++;
  },
  DECREMENT: function(state) {
    state.count--;
  }
},
```

VUEX - ACTIONS

Les actions gèrent la logique et commitent des changements d'état
Equivalent aux **méthodes** dans un composant
Des appels backend peuvent être effectués ici

```
    actions: {
      incrementAction: function(context) {
        console.log('Action
d\'incrémentation');
        context.commit("INCREMENT");
      },
      decrementAction: function(context) {
        console.log('Action de
décrémentation');
        context.commit("DECREMENT");
      }
    },
```

VUEX - ACTIONS

Les actions sont appelées depuis un composant via **dispatch**

```
store.dispatch("incrementAction");
```

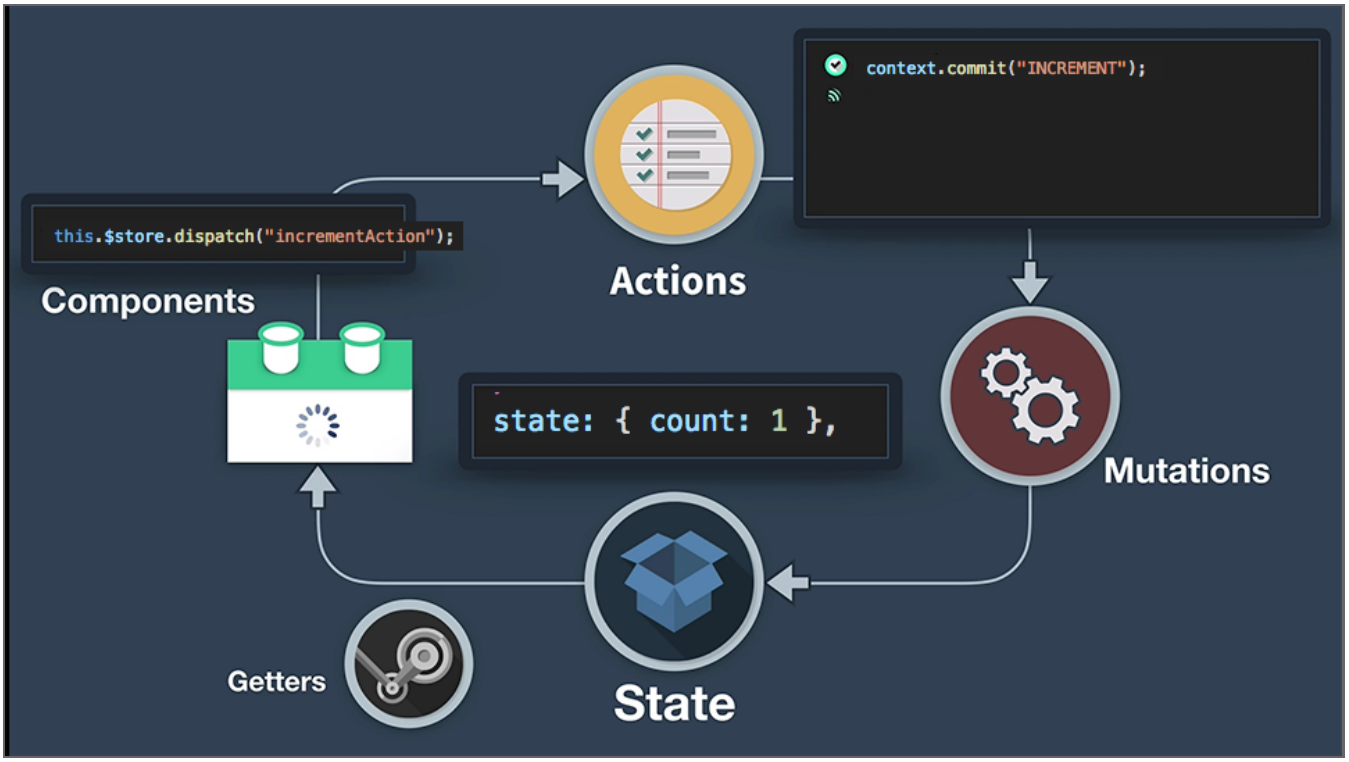
VUEX - GETTERS

Les getters permettent d'accéder en lecture aux états du store

```
getters: {
  count: function(state) {
    return state.count;
  }
}
```

```
store.getters.count;
```

VUEX


context.commit("INCREMENT");
this.$store.dispatch("incrementAction");
Components
Actions
state: { count: 1 },
Mutations
Getters
State

SOCKET.IO

socket.io

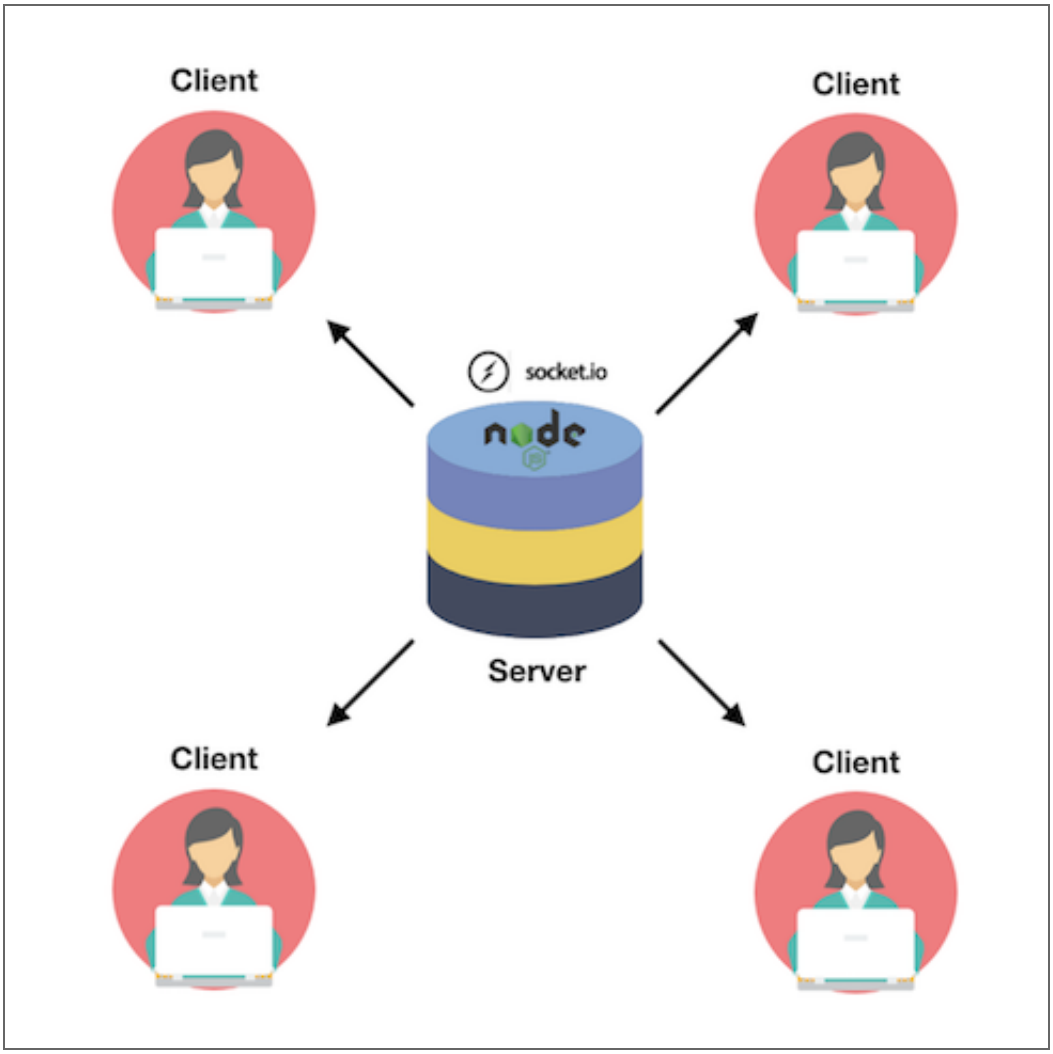
SOCKET.IO

Lors de nos TP, nous avons utilisé l'architecture Client Serveur
Les appels se font de manière asynchrone et c'est toujours le Client qui déclenche la communication

SOCKET.IO

Dans un système de chat, il serait intéressant d'avoir les messages en temps réel
Pourquoi ne pas utiliser un système de **stream** synchrone ?

SOCKET.IO



SOCKET.IO

Quatre choses à faire :

- Démarrer un stream
- Connecter les clients en écoute sur le stream
- Emettre des évènements
- Réagir aux évènements

SOCKET.IO

Démarrer un stream (depuis le serveur)

```
import { Server } from 'socket.io';
const io = new Server(server);

io.on('connection', socket =>
  console.log('Connection ' + socket.id)
);
```

SOCKET.IO

Connecter les clients (depuis le client)

```
import io from 'socket.io-client';
const socket = io('localhost:3000');
```

SOCKET.IO

Émettre des évènements (depuis le serveur)

```
io.emit('MESSAGE_ADDED', message);
```

SOCKET.IO

Réagir aux évènements (depuis le client)

```
socket.on('MESSAGE_ADDED', message => {
  messages.push(message);
});
```

CONCLUSION

LES MÊMES BESOINS...

Tous les frameworks JavaScript ont les mêmes besoins

- Gestion de composants (séparation des fonctionnalités, réutilisabilité)
- Gestion des routes
- Gestion des états

... AVEC DES LIBRAIRIES DIFFÉRENTES ...

Chaque framework a donc un ensemble de librairies pour répondre à ces besoins :

- **VueJS** : vue-loader (natif), vue-router, vuex
- **AngularJS** : composants (natif), angular-router (natif), ngrx, akita
- **ReactJS** : composants (natif), react router (natif), redux

... MAIS LES MÊMES CONCEPTS

Les concepts vus avec VueJS se retrouvent dans la plupart des frameworks :

- Composants et cycle de vie
- Data down, event up
- Directives
- 2 ways binding

Si vous maitrisez un framework, le passage à un autre peut être **rapide**.

Speaker notes